

Haswell Build Notes for Wed July 15 2026

Starting from a fresh install of 24.04.4. Haswell's trio of GTX 1080 TIs need:

```
nvidia-smi
| NVIDIA-SMI 570.211.01      Driver Version: 570.211.01      CUDA Version
: 12.8      |
```

```
nvidia-smi --query-gpu=gpu_name,compute_cap --format=csv
```

Ubuntu Nvidia Driver-less Installation Issues

I had to modify the hidden GRUB files on the hard drive, and force the system to always display the boot menu.

1. Install minimal, driver-less Ubuntu on hard drive from USB
2. Plug your Ubuntu installation USB back in. Boot from the USB and select "Ubuntu (safe graphics)" to get back into the live desktop environment.
3. Use the Terminal to Access Hard Drive. Find your Ubuntu installation partition:

```
sudo fdisk -l
```

4. Look for your internal drive (usually something like `/dev/sda2` or `/dev/nvme0n1p2`) formatted as Linux filesystem.

Mount your hard drive to the live environment (replace `sdXX` with your actual partition name, like `sdb2`):
`bash sudo mount /dev/sdXX /mnt` 5. Bind the core system directories so you can interact with it:

```
for i in /dev /dev/pts /proc /sys /run; do sudo mount --bind $i /mnt$i; done
```

6. Log into your hard drive's operating system via the terminal:

```
sudo chroot /mnt
```

7. Modify the Bootloader Configuration

```
nano /etc/default/grub
```

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=10
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash nomodeset"
```

8. Save and exit: Press Ctrl + O, hit Enter to confirm, then press Ctrl + X to exit the text editor.
9. Apply the configuration changes to the bootloader:

```
update-grub
```

10. Type exit to leave the hard drive environment, shut down the computer, unplug the USB, and turn it back on.

Install Nvidia driver (Haswell)

Before installing anything new, update all the libraries in this fresh install.

```
# upgrade all the packages in the minimal installation
sudo apt upgrade -y
```

Important Note: This is the last time to ever call `apt upgrade` because doing so from here on out will crash the dependencies - I'm working on a fix for this.

Register the CUDA keyring:

```
# register the CUDA keyring
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64
/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
```

Install NVidia driver and CUDA Toolkit 12.8

```
# Haswell needs Compute Architecture sm_61 for the 1080 TIs
# 570 (closed) was chosen arbitrarily over 570-open. Supposedly it is a little
  better at handing multiple displays.
sudo apt install nvidia-driver-570
sudo apt install cuda-toolkit-12-8
reboot
```

```
sudo apt install git gh # git tools
sudo snap install code --classic # vc code

# git config
git config --global user.name <user name>
git config --global user.email <email>
```

Inside `~/.bashrc`,

```
...
export CUDA_PATH=/usr/local/cuda
export PATH=$CUDA_PATH/bin
export LD_LIBRARY_PATH=$CUDA_PATH/lib64
```

UCX

Collect the code,

```
mkdir repos && cd repos
git clone --recursive https://github.com/openucx/ucx
cd ucx
```

Prerequisites

```
sudo apt update
sudo apt install build-essential automake autoconf pkg-config libtool
```

```
./autogen.sh
```

Issues with `src/tools/perf` Folder and Compute Architecture `sm_61`

`src/tools/perf` just contains a testing application, but its code contains references to features found in later hardware, so much of it fails to build for compute architecture `sm_61`. So prior to running `autogen.sh`, edit the `configure.ac` file and remove the references to `src/tools/perf` (line 300 in the current branch). Also remove the reference to `src/tools/perf` folder in the top-level `Makefile.am` file (line 49).

Configure UCX

Now, configure without `test-app` or `gtest`. Thus, `make` should totally ignore the `src/tools/perf` folder now.

```
./configure --prefix=/opt/ucx --with-cuda=/usr/local/cuda --with-nvcc-gencode="
-gencode=arch=compute_61,code=sm_61" --enable-mt --disable-test-app --
disable-gtest
make -j$(nproc)
sudo make install
```

For Threadripper this becomes:

```
# For Threadripper:
./configure --prefix=/opt/ucx --with-cuda=/usr/local/cuda --with-nvcc-gencode="
-gencode=arch=compute_86,code=sm_86 -gencode=arch=compute_120,code=sm_120"
--enable-mt
```

OpenMPI

Most of the first part of this build matches up with `builds.md`. This might be a good location to merge these documents. The `sudo apt install` prerequisites are more up to date in this document.

```
sudo apt install perl python3 python3-pip virtualenv flex bison libevent-dev
libhwloc-dev gfortran
```

Make sure to add the `--enable-sphinx` flag on all the calls to `./configure`, and enclose it in `source venv/bin/activate` and `deactivate` commands.

OpenPMIx with Documentation

```
cd 3rd-Party/openpmix
source venv/bin/activate # you should see (venv) to the left of the prompt
./configure --prefix=/opt/pmix --enable-sphinx
deactivate
make -j$(nproc)
sudo make install
```

PRRTE with Documentation

```
cd 3rd-Party/prrte
source venv/bin/activate # you should see (venv) to the left of the prompt
./configure --prefix=/opt/prrte --with-pmix=/opt/pmix --enable-sphinx --with-
sphinx=$VIRTUAL_ENV/bin/sphinx-build
deactivate
make -j$(nproc)
sudo make install
```

OpenMPI with Documentation

```
cd ../
./autogen.pl
source venv/bin/activate
```

```
./configure --prefix=/opt/mpi --with-libevent=external --with-hwloc=external
--with-pmix=/opt/pmix --with-prrte=/opt/prrte --with-ucx=/opt/ucx --with-
ucx-libdir=/opt/ucx/lib --with-cuda=/usr/local/cuda --with-cuda-libdir=/usr
/local/cuda/lib64/stubs --enable-sphinx --with-sphinx=$VIRTUAL_ENV/bin/
sphinx-build
deactivate
make -j$(nproc)
sudo make install
```

If building the documentation, don't forget to change the `manpath.config` file as in `builds.md` and update `mandb`:

```
sudo mandb
```

At this point, we're all caught up with `builds.md`, up to the ParaView build.

OpenFOAM .com Build Notes

Current Version ParaView Build Notes

I went with the current version of ParaView after trying the version of ParaView tarballed on OpenFOAM.com site. There were multiple reasons for this that mostly boiled down to brittle co-dependencies between VTK, QTs 5 and 6 and the only ParaView-aware OpenFOAM plugin, PVFoamReader. PVFoamReader is also the only tool that cares about htf5 support.

Pre-Requisites

```
sudo apt install cmake ninja-build libtbb-dev mesa-common-dev mesa-utils
freeglut3-dev xsltproc libxkbcommon-dev qt6-5compat-dev qt6-base-dev qt6-
tools-dev qt6-svg-dev
```

```
cd repos
git clone --recursive https://github.com/Kitware/ParaView.git
cd ParaView
mkdir paraview-build && cd paraview-build
cmake -GNinja -DCMAKE_INSTALL_PREFIX=/opt/paraview -DPARAVIEW_USE_PYTHON=ON -
DPARAVIEW_USE_CUDA=ON -DPARAVIEW_USE_MPI=ON -DVTK_SMP_IMPLEMENTATION_TYPE=
TBB -DCMAKE_CUDA_ARCHITECTURES=61 -DCMAKE_BUILD_TYPE=Release ../

# ninja -j will run out of memory at some point, and has to be re-run a couple
of times.
# expect scant feedback as some libraries take several minutes to build
ninja -j$(nproc)
sudo ninja install
```

Umpire

Umpire is used by Hypr and is a configurable install in their build.

Collect Umpire Code

```
git clone --recursive https://github.com/LLNL/Umpire.git
cd Umpire
mkdir build && cd build
```

Sphinx Documentation Revisited

For Umpire, we are back to wrapping configuration/cmake details within `activate/deactivate` blocks for the Umpire build.

```
sudo apt install doxygen python3-sphinx python3-sphinx-rtd-theme

# in each root directory where you need sphinx configured,
cd repos/my_tool

# create venv
python3 -m virtualenv venv --system-site-packages

# activate
source venv/bin/activate
pip install -r ../docs/requirements.txt

# ./configure --enable-sphinx, cmake, or whatever else requires sphinx
deactivate
```

Hash Definition Incomplete for GCC 13

However, you need to fix one small bug to get Umpire to build in GCC 13 with its strict compliance for C++20 (Umpire's codebase is lax on those rules). On line 27 of `src/umpire/strategy/ResourceAwarePool.hpp`, add the following code:

```
namespace std {
    template<>
    struct hash<umpire::resources::v1::Resource> {
        std::size_t operator()(const umpire::resources::v1::Resource& r) const
        noexcept {
            return std::hash<void*>{}((void*)&r);
        }
    };
}
```

Cmake Configuration and Build

```
cd umpire
mkdir build && cd build

# with documentation (do not use)
python3 -m virtualenv venv --system-site-packages
source venv/bin/activate
pip install -r ../docs/requirements.txt
cmake -DENABLE_MPI=ON -DENABLE_CUDA=ON -DBLT_ENABLE_CUDA=ON -DBLT_ENABLE_OPENMP
=ON -DBLT_ENABLE_MPI=ON -DUMPIRE_ENABLE_CUDA=ON -DUMPIRE_ENABLE_OPENMP=ON -
DUMPIRE_ENABLE_MPI=ON -DUMPIRE_ENABLE_IPC_SHARED_MEMORY=ON -
DUMPIRE_ENABLE_MPI3_SHARED_MEMORY=ON -DCMAKE_BUILD_TYPE=Release -
DCMAKE_CUDA_ARCHITECTURES=61 -DUMPIRE_ENABLE_C=ON -DCMAKE_INSTALL_PREFIX=/
opt/umpire ../
deactivate

cmake -DENABLE_MPI=ON -DENABLE_CUDA=ON -DBLT_ENABLE_CUDA=ON -DBLT_ENABLE_OPENMP
=ON -DBLT_ENABLE_MPI=ON -DBUILD_SHARED_LIBS=ON -DUMPIRE_ENABLE_CUDA=ON -
DUMPIRE_ENABLE_OPENMP=ON -DUMPIRE_ENABLE_MPI=ON -
```

```
DUMPIRE_ENABLE_IPC_SHARED_MEMORY=ON -DUMPIRE_ENABLE_MPI3_SHARED_MEMORY=ON -
DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=61 -DUMPIRE_ENABLE_C=
ON -DCMAKE_INSTALL_PREFIX=/opt/umfire ../
```

```
make -j$(nproc)
sudo make install
```

Include LD_LIBRARY_PATH and PATH changes in ~/.bashrc:

```
export PATH=/opt/paraview/bin:/opt/umfire/bin:/opt/mpi/bin:/opt/prt/bin:/opt
/pmix/bin:/opt/ucx/bin:$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=/opt/paraview/lib:/opt/umfire/lib:/opt/mpi/lib:/opt/
prt/lib:/opt/pmix/lib:/opt/ucx/lib:/opt/ucx/lib/ucx:$CUDA_HOME/lib64
```

Hypre

```
cd repos
git clone --recursive https://github.com/hypre-space/hypre.git
cd hypre
mkdir build && cd build
cmake -DBUILD_SHARED_LIBS=ON -DCMAKE_POSITION_INDEPENDENT_CODE=ON -
DCMAKE_BUILD_TYPE=Release -DCMAKE_C_COMPILER=mpicc -DCMAKE_CXX_COMPILER=
mpicxx -DHYPRE_ENABLE_CUDA=ON -DMPI_INCLUDE_PATH=/opt/mpi/include -
DCMAKE_CUDA_ARCHITECTURES=61 -DHYPRE_ENABLE_GPU_AWARE_MPI=ON -
DHYPRE_ENABLE_OPENMP=ON -DHYPRE_ENABLE_UMPIRE=ON -DHYPRE_ENABLE_UMPIRE_HOST
=ON -DHYPRE_ENABLE_UMPIRE_DEVICE=ON -DCMAKE_INSTALL_PREFIX=/opt/hypre -
DTPL_UMPIRE_INCLUDE_DIRS=/opt/umfire/include -DTPL_UMPIRE_LIBRARIES=/opt/
umfire/lib/libumfire.so ../src
make -j$(nproc)
sudo make install
```

Scotch

```
# Need Scotch for orthogonal decomposition
git clone --recursive https://gitlab.inria.fr/scotch/scotch.git
```

PETSc

```
# Need PETSc for Scientific Computation
git clone --recursive https://github.com/petsc/PETSc.get
```

```
# PETSc config attempt
./configure --prefix=/opt/petsc --with-mpi-dir=/opt/mpi --with-hypre-dir=/opt/
hypre --with-cuda=1 --with-hypre-gpu=1
```

OpenFOAM .com Build Notes

Clone the repos - ThirdParty-common is of questionable value and may not be useful, given all the external tool configuration that is required.

Put OpenFOAM and the ThirdParty-common repositories into a single repo for convenience.

```
cd repos
mkdir OpenFOAM_com && cd OpenFOAM_com
git init -b main
gh repo create OpenFOAM_com --private --source=. --remote=origin
git submodule add https://gitlab.com/openfoam/core/OpenFOAM.git
git submodule add https://gitlab.com/openfoam/core/ThirdParty-common.git
git add .
git commit -m "Added OpenFOAM.com submodules"
git push -u origin main
```

ExaFoam/AmgX4Foam

Build Notes for Sunday, July 19

ADIOS2

– ignore, not used by system

ccmio

– ignore, not used by system

CGAL

```
sudo apt update
sudo apt install libcgall-dev libgmp-dev libmpfr-dev libboost-system-dev
```

Inside prefs.sh

```
#can use default cgal
export CGAL_ARCH_PATH=/usr
```

FFTW

```
# Prerequisites
sudo apt install texinfo

./configure --prefix=/opt/fftw --enable-shared --enable-threads --enable-openmp
--enable-mpi --enable-sse2 --enable-avx --enable-avx2 LDFLAGS="-Wl,-rpath
, /opt/fftw/lib"

make -j$(nproc)
sudo make install
```

Hypre

– already built export HYPREARCHPATH=/opt/hypre

hdf5

– just ignore, not building paraview foam reader module

KaHIP

```
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=/opt/kahip -DCMAKE_BUILD_TYPE=Release -
      DBUILD_SHARED_LIBS=ON -DCMAKE_C_COMPILER=mpicc -DCMAKE_CXX_COMPILER=mpicxx
      -DCMAKE_INSTALL_LIBDIR=/opt/kahip/lib -DCMAKE_INSTALL_RPATH=/opt/kahip/lib
..
make -j$(nproc)
sudo make install
```

KarypisLabs Suite (GKlib, METIS, ParMETIS)

Build a consolidated repository containing MEITS, ParMETIS, and the GKLib dependency

```
cd repos
mkdir KarypisLabs && cd KarypisLab
git init -b main
gh repo create KarypisLab --private --source=. --remote=origin
git submodule add https://github.com/KarypisLab/GKlib.git
git submodule add https://github.com/KarypisLab/METIS.git
git submodule add https://github.com/KarypisLab/ParMETIS.git
git add .
git commit -m "Added KarypisLab tools GKLib, Metis, and ParMetis"
git push -u origin main
```

GKlib

```
cd GKlib
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=/opt/karypis -DCMAKE_BUILD_TYPE=Release -DSHARED=
      ON -DOPENMP=ON -Di64=0 -DCMAKE_INSTALL_LIBDIR=/opt/karypis/lib -
      DCMAKE_INSTALL_RPATH=/opt/karypis/lib ..
make -j$(nproc)
sudo make install
```

METIS

```
cd METIS
mkdir metis-build && cd metis-build
cmake -DCMAKE_BUILD_TYPE=Release -DSHARED=ON -DOPENMP=ON -DCMAKE_C_FLAGS="-
      march=native -O3 -DIDXTYPEWIDTH=32 -DREALTYPEWIDTH=32" -
      DCMAKE_INSTALL_LIBDIR=/opt/karypis/lib -DCMAKE_INSTALL_PREFIX=/opt/karypis
      -DGKLIB_PATH=/opt/karypis -DCMAKE_INSTALL_RPATH=/opt/karypis/lib ..
make -j$(nproc)
sudo make install
```

ParMETIS

```
cmake -DCMAKE_BUILD_TYPE=Release -DSHARED=ON -DOPENMP=ON -DCMAKE_INSTALL_PREFIX
      =/opt/karypis -DCMAKE_INSTALL_RPATH=/opt/karypis/lib -DGKLIB_PATH=/opt/
      karypis -DMETIS_PATH=/opt/karypis -DCMAKE_C_COMPILER=mpicc -DCMAKE_C_FLAGS=
      "-DIDXTYPEWIDTH=32 -DREALTYPEWIDTH=32" ..
```


mgridgen

– just ignore, legacy

MPI

Inside prefs.sh

```
# already built
export MPI_ARCH_PATH=/opt/mpi
```

ParaView

Inside prefs.sh

```
# already built
export ParaView_VERSION="none"
```

ParaView-system

– just ignore, nothing to do

PETSc

```
cd petsc
mkdir build
./configure --with-hwloc=1 --with-boost=1 --with-ptscotch=1 --with-ptscotch-dir
=/opt/scotch --with-amgx-dir=/opt/amgx --with-petsc-arch=build --prefix=/
opt/petsc --with-debugging=0 --with-shared-libraries=1 --with-mpi-dir=/opt/
mpi --with-scotch-dir=/opt/scotch --with-zoltan-dir=/opt/zoltan --with-
cuda=1 --with-cuda-dir=/usr/local/cuda --with-umpire=1 --with-umpire-
include=/opt/umpire/include --with-umpire-lib=/opt/umpire/lib/libumpire.so
--with-hypre=1 --with-hypre-include=/opt/hypre/include --with-hypre-lib="/
opt/hypre/lib/libHYPRE.so_/_opt/umpire/lib/libumpire.so_/_opt/umpire/lib/
libcamp.so_/_opt/umpire/lib/libfmt.so_/_usr/local/cuda/lib64/libcudart.so_/_
usr/local/cuda/lib64/libcublas.so_/_usr/local/cuda/lib64/libcusparse.so" --
with-metis-include=/opt/karypis/include --with-metis-lib="/opt/karypis/lib/
libmetis.so_/_opt/karypis/lib/libGKlib.so" --with-parmetis-include=/opt/
karypis/include --with-parmetis-lib="/opt/karypis/lib/libparmetis.so_/_opt/
karypis/lib/libmetis.so_/_opt/karypis/lib/libGKlib.so" CUDAFLAGS+="-arch=
sm_61" CUDAOPTFLAGS="-O3" COPTFLAGS="-O3_/_march=native" CXXOPTFLAGS="-O3_/_
march=native" FOPTFLAGS="-O3_/_march=native" --with-openmp --with-fortran-
bindings=1 --with-fortran-kernels=0 --with-fortranlib-autodetect=0
```

Inside prefs.sh

```
export PETSC_ARCH_PATH=/opt/petsc
```

Scotch

```
cd Scotch
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=ON -DCMAKE_INSTALL_PREFIX
      =/opt/scotch -DCMAKE_INSTALL_LIBDIR=/opt/scotch/lib -DCMAKE_INSTALL_RPATH=/
      opt/scotch/lib -DINSTALL_METIS_HEADERS=OFF -DMPI_THREAD_MULTIPLE=ON -
      DCMCMAKE_C_COMPILER=mpicc ..
make -j$(nproc)
sudo make install
```

Inside prefs.sh

```
export SCOTCH_ARCH_PATH=/opt/scotch
```

Umpire

– already built export UMPIREARCHPATH=/opt/umpire

VTK

export VTK_VERSION = "none"

Zoltan

```
cd Zoltan
mkdir build && cd build
../configure --prefix=/opt/zoltan --enable-mpi CC=mpicc CXX=mpicxx --with-
  parmetis-incdir=/opt/karypis/include --with-parmetis-libdir=/opt/karypis/
  lib --with-scotch-incdir=/opt/scotch/include --with-scotch-libdir=/opt/
  scotch/lib CFLAGS="-O3 -fPIC" CXXFLAGS="-O3 -fPIC" --with-scotch --with-
  parmetis CPPFLAGS="-I/opt/karypis/include -DIDXTYPEWIDTH=32 -DREALTYPEWIDTH
  =32" LDFLAGS="-L/opt/karypis/lib"
make everything -j$(nproc)
sudo make install
```

Post-Game Build Order

- UCX
- OpenPMIx
- PR RTE
- OpenMPI
- ParaView
- Umpire
- Hypre
- FFTW
- KaHIP
- GKlib
- METIS
- ParMETIS
- Scotch
- Zoltan

- AMGX
- PETSc
- OpenFOAM
- AmgX4Foam

AMGX Revisited

```
cd amgx
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/opt/amgx -
      DCMCMAKE_CUDA_ARCHITECTURES=61 -DCMAKE_C_COMPILER=mpicc -DCMAKE_CXX_COMPILER=
      mpicxx -DCMAKE_CUDA_FLAGS="-I/opt/mpi/include" -DCMAKE_EXE_LINKER_FLAGS="-
      L/opt/mpi/lib_lmpi_lmpi_cxx" ..
make -j$(nproc)
sudo make install
```

Had some issues with test launcher using mpi compilers. Ended up changing line 55 in `src/CMakeLists.txt` prior to running `cmake`:

```
target_link_libraries(amgx_tests_launcher "/opt/mpi/lib/libmpi.so")
```